$$5.\ Array\ Address\ Calculation[Base\ Address + Index\ \times SizeOf(DataType)]$$

$$As\ we\ see\ on:\quad getting \rightarrow$$

$$return\ stack[top];$$

==========

$TEXT\ /\ CODE\ SEGMENT\ (Where\ the\ Program\ Counter\ points)$

==========

$(The\ PC\ walks\ down\ these\ addresses\ one\ by\ one)$

$[\ 0x1000\ ] -> \ [\ Instruction: LOAD\ 'top'\ into\ Reg1\ ]$

$[\ 0x1004\ ] -> \ [\ Instruction: CALCULATE\ address\ of\ stack[top]\ ]$

$[\ 0x1008\ ] -> \ [\ Instruction: READ\ data\ from\ array\ into\ Reg2\ ]$

==========

$TEXT\ /\ CODE\ SEGMENT\ (Instruction\ Memory)$

==========

$[\ Address\ ]\quad [\ 16-bit\ Machine\ Code\ ]\quad [\ CU\ Translation\ (Invisible)\ ]$

$[\ 0x1000\ ] -> \ [\ 0100\ 100000000000\ ]\ <--\ Opcode: 0100\ (LOAD), Operand: 0x800$

$[\ 0x1004\ ] -> \ [\ 0111\ 000001010000\ ]\ <--\ Opcode: 0111\ (CALC), Operand: Base\ Array\ Addr: 0x050$

$[\ 0x1008\ ] -> \ [\ 0100\ 000000000000\ ]\ <--\ Opcode: 0100\ (LOAD), Operand: (Calculated\ Addr)$

$1st\ 0100(Load)\ then: 1000_2 = 0x8_{16}, 0000_2 = 0x0_{16}, 0000_2 = 0x0_{16} \Rightarrow 0x800, its\ the\ address\ where\ value\ of\ variable\ TOP\ is\ stored, say\ TOP = 4\ stored\ at\ 0x800\ and\ hence\ 4\ loaded\ from\ the\ correspondence\ address: 0x800.$

$2nd\ 0111(Calc)\ then:\ 0000_2 = 0x0_{16}, 0101_2 = 0x5_{16}, 0000_2 = 0x0_{16} \Rightarrow 0x050, Calculate\ address\ of\ stack[top], i.e.\ from\ Base\ Address: 0x050_{16}.$

==========

$THE\ ARRAY\ CALCULATION\ (Inside\ the\ ALU)$

==========

$[\ Operand: 0x050\ ]\ ----------> [\ Base\ Address: 0x050\ ]$

$\downarrow$

$[\ Reg1\ (holds\ index\ '4')\ ]\ --------> [\ Index: 4\ ] * [\ Size\ of\ int: 4\ ]$

$\downarrow$

$[\ ALGORITHM\ AND\ LOGIC\ UNIT\ ]\ <------ Performs: 0x050 + (4 * 4)$

$\downarrow$ $(0x050 + 16\ in\ decimal)$

$(0x050 + 0x10\ in\ hex)$

$[\ Final\ Array\ Address: 0x060\ ]$

$Lets\ deduce\ REG1\ of\ CPU\ ,strictly\ holds\ raw\ number: 4\ (index).$

$Its\ doesnot\ multiply\ as\ shown\ above\ rather, it\ occurs\ like\ this\ as\ shown\ below:$

1. The Compiler's Job (Choosing the Recipe)

When we wrote the C++ code for an `int` array, the compiler knew that an integer takes up 4 bytes.

Because the compiler knows this, it deliberately selects the specific machine code Opcode `0111`. In modern CPU design, an opcode like this (often called a "scaled index memory operand") is specifically designed for arrays. The compiler writes `0111(CALC)` into the `.exe` file, which essentially means: *"Hey CPU, do an Add, but scale the index by 4 first."*

2. The Control Unit's Job (Executing the Recipe)

As the program is running and that `0111` reaches the Instruction Register (IR), the compiler's job is completely done. Now, the **Control Unit (CU)** is the absolute boss.

The CU reads the `0111` and looks up its internal "recipe book" (this is literally what **microcode** is—a hardwired list of electrical signals tied to a specific opcode).

Because the CU sees `0111`, its microcode tells it to fire a very specific sequence of electrical pulses to control the traffic:

- **Pulse 1 (To REG 1):** *"Open your output gates and release the `4`."*
- **Pulse 2 (To the Bit-Shifter / Scaler):** *"Wake up! Intercept the data on the wire and shift its bits to the left by 2 spaces."* (This is the CU physically commanding the multiplication).
- **Pulse 3 (To the ALU):** *"Take the Base Address from the instruction, take the new `16` coming out of the shifter, and ADD them together."*

==========

$\boldsymbol{THE\ CHAIN\ OF\ COMMAND}$

==========

$[\ \boldsymbol{C++Code}\ ] \text{ -> } \boldsymbol{int\ stack[top]};\ \ (\boldsymbol{Compiler\ sees\ 'int', knows\ it\ needs\ x4})$

$\downarrow$

$[\ \boldsymbol{Machine\ Code}\ ] \text{ -> } \boldsymbol{Opcode\ 0111}\ (\boldsymbol{Compiler\ chooses\ the\ "Scale\ by\ 4"\ opcode})$

$\downarrow$

$[\ \boldsymbol{Control\ Unit}\ ] \text{ -> } \boldsymbol{Decodes\ 0111}.$

$\downarrow$ $[\ \boldsymbol{Reads\ its\ microcode\ recipe.}\ ]$

$[\ \boldsymbol{Micro-Signals}\ ] \text{ -> } \boldsymbol{CU\ fires\ the\ exact\ electrical\ pulse\ to\ the\ Bit-Shifter}$

$\boldsymbol{to\ multiply\ the\ index\ before\ it\ hits\ the\ ALU}!$

When the compiler generates the `0111` (CALC) instruction, it actually bakes the "Size of int" right into the instruction's microcode. It tells the CPU: *"Hey, take whatever is in REG 1, multiply it by 4, and THEN add it to the Base Address."*

But hardware engineers don't like using full, slow multiplication circuits just to find memory addresses. Instead, they use a blazing-fast hardware trick called a **Bit Shift**. Multiplying any binary number by 4 is as simple as shifting its bits to the left by 2 spaces!

- `0000 0100` (Decimal 4, sitting in REG 1)
- *Shift Left by 2...*
- `0001 0000` (Decimal 16, instantly generated by the wires!)

Here is the corrected, ultra-precise diagram showing exactly where that multiplication happens physically:

$$\boldsymbol{THE\ TRUE\ HARDWARE\ ROUTING\ (The\ Address\ Generation\ Phase)}$$

$$[\ \boldsymbol{Operand}: \mathbf{0x050}\ ] - - -

$3rd\ 0100(LOAD)\ then:\ 0000_2, 0000_2, 0000_2 - When\ this\ reaches\ to\ Instruction\ Register(IR)$
$Control\ Unit\ splits\ the\ data\ reads\ OPCODE, 0100\ as\ LOAD\ and\ recognizes\ as\ INDIRECT$
$LOAD\ and\ ignores\ the\ 12\ bit\ operand: 0000\ 0000\ 0000\ completely.$

$The\ compiler\ cannot\ hardcode\ the\ address\ 0x060\ into\ the\ instruction\ because\ 0x060$
$is\ just\ a\ temporary\ result. If\ the\ top\ index\ changes\ to\ 3\ tomorrow, the\ address\ will\ be$
$different. The\ compiler\ puts\ 000000000000\ there\ purely\ as\ a\ dummy\ placeholder.$

$0000_2 = 0x0, 0000_2 = 0x0, 0000_2 = 0x0 \Longrightarrow 0x000 - this\ is\ a\ dummy\ placeholder.$
$Because\ the\ CU\ knows\ to\ ignore\ the\ 0x000\ placeholder, it\ turns\ its\ attention\ to\ the$
$calculated\ address\ (0x060) that\ we\ just\ pushed\ into\ the\ Memory\ Address\ Register\ (MAR)$
$in\ the\ last\ cycle.$

$The\ setup\ is\ now\ perfect: The\ MAR\ securely\ holds\ the\ correct\ map\ (0x060).$

$This\ occurs\ as\ Dynamic\ Runtime\ Calculation(often\ termed\ as\ ``Indirect\ Addressing``)$
$As\ dummy\ address\ is\ placed\ purposely\ to\ be\ ignored\ in\ the\ dynamic\ runtime\ mode.$

==========

$THE\ PRECISE\ INTERNAL\ ROUTING\ (ALU -> MAR)$

==========

[$CONTROL\ UNIT\ (CU)$]

|

| --- ($Signal\ 1$: "$Output\ Enable$")

| ------> $Sent\ to$ [$ALU\ Output\ Buffer$] : "$Release\ the\ 0x060$!"

|

| --- ($Signal\ 2$: "$Load\ Enable$")

| ------> $Sent\ to$ [MAR] : "$Catch\ the\ 0x060$!"

$- -$

$THE\ PHYSICAL\ PATHWAY\ (Internal\ Logic\ Gates)$

$- -$

$[\ ALU\ Output\ ]\ Holds: 0x060$

$\downarrow$ $(Flows\ across\ internal\ CPU\ wiring)$

$[\ Memory\ Addr.Reg\ (MAR)\ ]$

$(Load\ Enable\ gate\ opens, latches\ the\ address)$

$Now\ Holds: 0x060$

$==========$

$THE\ DATA\ TRANSFER\ THROUGH\ ADDRESS$

$==========$

$- -\ STEP\ 1: Using\ the\ Address\ - -$

$[\ ALU\ ]\ Calculates\ Address: 0x060$

$\downarrow$ $(Internal\ Routing)$

$[\ MAR\ ]\ Now\ Holds: 0x060\ \ <- -\ (The\ Map)$

$- -\ STEP\ 2: Fetching\ the\ Data\ - -$

$[\ RAM\ Slot\ , STACK\ ADDRESS: 0x060\ ]\ Holds\ Data: 30$

$\downarrow$ $(Via\ Data\ Bus\ and\ through\ MDR)$

$[\ Reg\ 2\ ]\ Now\ Holds: 30\ \ <- -\ (The\ Treasure!)$

$\boldsymbol{What\ happens\ during}: -$

$\boldsymbol{for}\ (\boldsymbol{int\ i} = \mathbf{0}; \boldsymbol{i} < \boldsymbol{n}; \boldsymbol{i}++)\{$

$\boldsymbol{cout} << "\boldsymbol{arr}[\boldsymbol{i}]" << \boldsymbol{endl};$

}

$\boldsymbol{return}\ \mathbf{0};$

$\boldsymbol{Lets\ say\ n} = \mathbf{1}.$

==========

RAM MEMORY DUMP: TEXT/CODE SEGMENT (The Loop)

==========

Address	*Raw Hex*	*Raw Binary*
$0x1000$:	$0x4100$	0100 0001 00000000
$0x1004$:	$0x4201$	0100 0010 00000001
$0x1008$:	$0x8120$	1000 0001 00100000
$0x100C$:	$0x901C$	1001 0000 00011100
$0x1010$:	$0xC000$	1100 0000 00000000 $<--$ (*Abstract 'PRINT' call*)
$0x1014$:	$0x2101$	0010 0001 00000001
$0x1018$:	$0xF008$	1111 0000 00001000
$0x101C$:	$0x4300$	0100 0011 00000000
0x1020:	0x3F00	0011 1111 00000000

$\boldsymbol{Here\ is\ the\ secret\ decoder\ ring\ the\ compiler\ uses}$:

- $\boldsymbol{LOAD\ Opcode} = \mathbf{0100}\ (\boldsymbol{Hex}\ \mathbf{4})$
- $\boldsymbol{COMPARE\ Opcode} = \mathbf{1000}\ (\boldsymbol{Hex}\ \mathbf{8})$
- $\boldsymbol{JUMP_IF_EQ\ Opcode} = \mathbf{1001}\ (\boldsymbol{Hex}\ \mathbf{9})$
- $\boldsymbol{ADD\ Opcode} = \mathbf{0010}\ (\boldsymbol{Hex}\ \mathbf{2})$
- $\boldsymbol{JUMP\ Opcode} = \mathbf{1111}\ (\boldsymbol{Hex}\ \boldsymbol{F})$
- $\boldsymbol{REG}\ \mathbf{1} = \mathbf{0001}$
- $\boldsymbol{REG}\ \mathbf{2} = \mathbf{0010}$
- $\boldsymbol{REG}\ \mathbf{3} = \mathbf{0011}$
- $\boldsymbol{Program\ Counter} = \mathbf{1111}$
- $\boldsymbol{POP} = \mathbf{0011}$

$\boldsymbol{The\ 16-bit\ Blueprint}$

Every single instruction in our TEXT segment is exactly 16 *bits long,*

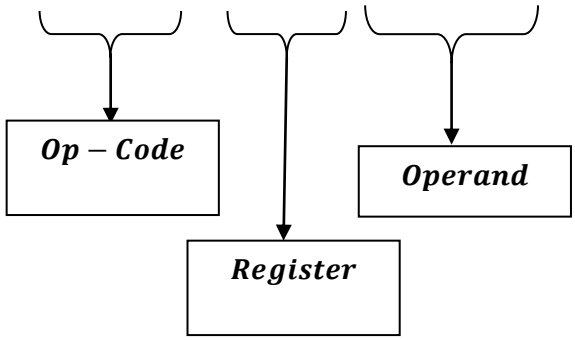
chopped into three distinct sections:

- [$\boldsymbol{Bits\ 15-12}$] $\boldsymbol{Opcode}$: *The* $4-$ *bit command* (*What to do*).
- [$\boldsymbol{Bits\ 11-8}$] $\boldsymbol{Register}$: *The* $4-$ *bit target* (*Who to do it to*).
- [$\boldsymbol{Bits\ 7-0}$] $\boldsymbol{Operand}$: *The* $8-$ *bit data or memory address*.

$\boldsymbol{Lets\ see\ the\ first\ binary\ module}$:

15 14 13 12 11 10 9 8 7 6 5 4 3 2 1 0

0 1 0 0 0 0 0 1 0 0 0 0 0 0 0 0



$\mathbf{0100\ 0001\ 00000000} \rightarrow \boldsymbol{LOAD\ \ REG1}$ `0` $\boldsymbol{i.e.}$

$\mathbf{1.}\boldsymbol{The\ Setup}$ $(\boldsymbol{int\ i = 0, n = 1})$

- $\mathbf{0x1000}$: $\mathbf{0x4100} \rightarrow$ [0100] [0001] [00000000]

  - $\mathbf{0100}$: $\boldsymbol{The\ CU(Control\ Unit)\ reads\ "LOAD"}$.
  - $\mathbf{0001}$: $\boldsymbol{The\ CU(Control\ Unit)\ \ routes\ to\ REG\ 1}$.
  - $\mathbf{00000000}$: $\boldsymbol{The\ CU(Control\ Unit)\ pushes\ the\ raw\ number\ 0.(int\ i = 0)}$.

    [$\boldsymbol{Lower\ Bound}$]

$\mathbf{0100\ 0010\ 00000001} \rightarrow \boldsymbol{LOAD\ REG2}$ `1`

$\mathbf{0x1004}$: $\mathbf{0x4201} \rightarrow$ [0100] [0010] [00000001]

- $\mathbf{0100}$: "$\boldsymbol{LOAD}$"
- $\mathbf{0010}$: $\boldsymbol{Route\ to\ REG\ 2}$.
- $\mathbf{00000001}$: $\boldsymbol{Push\ the\ raw\ number\ 1}$. ($\boldsymbol{This\ is\ , n = 1}$)[$\boldsymbol{Upper\ Bound}$]

$\mathbf{1000\ 0001\ 00100000} \rightarrow \boldsymbol{COMPARE\ REG1\ REG2}$

$\mathbf{2}.\boldsymbol{The\ Condition\ Check}\ (i < n)$

- $\mathbf{0x1008}$: $\mathbf{0x8120} \rightarrow$ [1000][0001][0010][0000]
  - $\mathbf{1000}$: $\boldsymbol{The\ CU\ reads\ "COMPARE"\ (Trigger\ the\ ALU\ subtractor)}$.
  - $\mathbf{0001}$: $\boldsymbol{Send\ REG\ 1\ into\ ALU\ Input\ A}$.
  - $\mathbf{0010}$: $\boldsymbol{Send\ REG\ 2\ into\ ALU\ Input\ B}$.

($The\ last\ 4\ zeros\ are\ empty\ padding\ because\ the\ ALU\ does\ all\ the\ work$).

$\mathbf{1100\ 0000\ 00000000} \rightarrow \boldsymbol{Here\ 1100\ is\ OUT\ or\ SYSCALL\ , it\ tells\ CU(Control\ Unit)}$

"$\boldsymbol{We\ are\ leaving\ the\ CPU\ and\ talking\ to\ an\ external\ device}$."

$\mathbf{0000} \rightarrow (\boldsymbol{The\ Device\ ID})$: $\boldsymbol{The\ CU\ uses\ this\ 4-bit\ chunk\ to\ know\ which\ device}$

$\boldsymbol{to\ talk\ to. Let's\ say\ 0000\ means\ the\ Monitor, 0001\ means\ the\ Printer}$

$\boldsymbol{, and\ 0010\ means\ the\ Hard\ Drive}$.

$\mathbf{00000000}\ (\boldsymbol{The\ Data\ Address})$: $\boldsymbol{This\ is\ the\ 8-bit\ pointer. It\ tells\ the\ CU}$:

"$\boldsymbol{Go\ look\ at\ memory\ address\ 0x00\ in\ the\ DATA\ Segment\ to\ find}$

$\boldsymbol{the\ text\ you\ need\ to\ print}$."

0010 0001 00000001 → ***ADD REG*1 `1`**

The Increment $(i++)$

- $\mathbf{0x1014}$: $\mathbf{0x2101}$ → [0010][0001][00000001]
  - **0010**: ***The CU reads "ADD"***.
  - **0001**: ***Target REG* 1**.
  - **00000001**: ***The raw number*** 1. (***This forces REG* 1 *to go from* 0 *to* 1**).

1111 0000 00001000 – ***UNCONDITIONAL JUMP*** (***ignore register***) ***JAM* 0*x*1008 *and***

Force loop to start.

The Loop – Back (***JUMP***)

- $\mathbf{0x1018}$: $\mathbf{0xF008}$ → [1111] [0000] [00001000]
  - 1111: "*UNCONDITIONAL JUMP*" (*Ignore all flags, just teleport*).
  - 0000: (*Ignore register*).
  - 00001000: *This is decimal* 8. *In Hex, it is* 0*x*08. *The CU violently jams* 0*x*1008 *into the Program Counter to force the loop to restart*.

0100 0011 00000000 – ***return* 0**; –***LOAD REG*3 `0`**

→ ***It loads* `0` *into the output register*(*Success Code*)*i.e. at REG* 3**

[***OUTPUT REGISTER – REGISTER which hold the ouput during ALU operation like*** : ***Accumulator***]

0011 1111 00000000 – [POP] [Program – Counter] [0 – Padding]

The Control Unit (CU) executes our final 0*x*3F00 (*NATIVE POP to the PC*) *instruction at the end of* main(), *function.*

→ ***Performs a Native Hardware POP operation***.

→ ***CPU puts the current stack pointer address and puts it to MAR***.

→ ***MAR broadcast this address to the ADDRESS BUS*** .

→ ***MDR catches the Return Address stored at the stack address through the data bus***.

→ ***Now Return address is now safely inside MDR***.

→ ***CU promotes this return address from MDR to Program Counter***.

→ ***Now CPU teleports back to return address, to resume the program***.

$\mathbf{0x1020}$: $\mathbf{0x3F00} \rightarrow$ [0011][1111][00000000]

- [**0011**](***The Opcode***): *This is the native POP command. It tells the Control Unit (CU) to grab whatever is sitting at the very top of the Hardware Stack.*

- [**1111**](***The Target***): *This is the internal hardware ID for the* ***Program Counter*** (***PC***). *It tells the CU: "Do not put this data into a normal math register like REG 1 or REG 2. Route this data directly into the PC!"*

- [**00000000**](***The Padding***): *This is completely blank. Why? Because a POP instruction doesn't need an address operand! It automatically relies on the Stack Pointer (SP) to know exactly where to look in memory.*

The Physical Execution (The Final Clock Cycles)

When the CPU reads 0x3F00 at the very end of your program, here is the final sequence of electrical signals the CU fires:

1. ***The Read***: *The CU routes the Stack Pointer's address into the MAR, fires the READ signal, and pulls the Operating System's return address out of the RAM and into the MDR.*

2. ***The Route***: *The CU sees the 1111 target in the instruction. It violently routes the return address from the MDR directly into the Program Counter.*

3. ***The Cleanup***: *The CU fires the signal to the hardware subtractor to adjust the Stack Pointer ($SP = SP - 4$), officially removing your program's data from the stack.*

==========

$\boldsymbol{THE\ FINAL\ TELEPORT\ (Returning\ to\ the\ OS)}$

==========

[$\boldsymbol{Stack\ Pointer\ (SP)}$]

↓

[$\boldsymbol{Memory\ Addr\ Reg\ (MAR)}$] $\boldsymbol{--->}$ $\boldsymbol{(Points\ to\ Top\ of\ Stack\ in\ RAM)}$

$\boldsymbol{(RAM\ finds\ OS\ Return\ Address: 0x8000)}$

↓

[$\boldsymbol{Memory\ Data\ Reg\ (MDR)}$] $\boldsymbol{<---}$ $\boldsymbol{(Data\ Bus\ carries\ 0x8000)}$ $\boldsymbol{----}$

↓

$\boldsymbol{(CU\ violently\ routes\ the\ breadcrumb\ to\ the\ PC!)}$

↓

[$\boldsymbol{Program\ Counter\ (PC)}$] $\boldsymbol{OVERWRITTEN!\ Now\ Holds: 0x8000}$

The Aftermath

The clock ticks one more time. The CPU looks at the Program Counter to figure out what to do next.

Because the PC now holds `0x8000`, the CPU fetches the instruction at `0x8000`. But `0x8000` is not in our `.exe` file anymore—it is in the Windows Operating System's memory space!

Instantly, Windows wakes back up. It looks at **REG 3** (where we left that `0` for success), sees that your program finished perfectly, and clears your program's data out of the RAM.

Phase 1: General Map of the Loop Flow

Before looking at the detailed wires, let's map out the four distinct "Zones" of the loop in memory. Notice that the Program Counter (PC) has two modes: **Normal Mode** (+4 sequence) and **Jump Mode** (teleporting).

```
==============================================
 THE COMPILED LOOP INSTRUCTION MEMORY - VISUAL "MAP"
==========================================

 [ Address ]                    [ Opcodes ]

[ ZONE 1: SETUP ]
-> [ 0x1000 ] ----> [ LOAD '0' REG1 (i) ] ---> NORMAL MODE (PC + 4)
-> [ 0x1004 ] ----> [ LOAD '1' REG2 (n) ]

[ ZONE 2: CONDITION ] <---------------- JUMP MODE BACKWARD
-> [ 0x1008 ] ----> [ COMPARE REG1, REG2 ]  (Evaluates i vs n)
-> [ 0x100C ] ----> [ JUMP_IF_EQUAL 0x101C ] (Escapes loop?)

[ ZONE 3: BODY ]
-> [ 0x1010 ] ----> [ PRINT "arr[i]" ] ----> NORMAL MODE (PC + 4)
-> [ 0x1014 ] ----> [ ADD '1' REG1 (i++) ]

[ ZONE 4: LOOP - BACK ]
-> [ 0x1018 ] ----> [ JUMP directly 0x1008 ] -> JUMP MODE FORWARD

[ ZONE 5: OUTSIDE ]
 -> [ 0x101C ] ---> [ (Next instruction) ] <--- JUMP MODE ESCAPE TARGET
```

Phase 2: Detailed Micro-operations and Wire Activity.

$Visual\ Scenario\ A$: $Fetching\ an\ Instruction\ (from\ 0x1008)$

$$==========$$

$$VISUAL\ A: FETCHING\ THE\ INSTRUCTION\ (the\ Fetch - Execute\ Cycle\ Start)$$

$$==========$$

$Clock\ Cycle\ 1\ (T1)$: $ADDRESS\ ROUTING$

[$Program\ Counter\ (PC)$] $Holds$: $0x1008\ (The\ current\ address)$

$\downarrow$ ($CU\ Signal\ 1$: PC_OUT)

($Flows\ over\ internal\ CPU\ wiring$)

[$Memory\ Address\ Register\ (MAR)$] ($CU\ Signal\ 2$: $MAR_IN\ latch$)

$Now\ Holds$: $0x1008$

$Clock\ Cycle\ 2\ (T2)$: $THE\ RAM\ READ$

[$Memory\ Address\ Register\ (MAR)$] -> [$ADDRESS\ BUS\ carries$: $0x1008$] -> [$RAM\ Decoder$]

|

[$CONTROL\ UNIT\ (CU)$] -> [$CONTROL\ BUS\ carries$: "$READ$"] -> [$RAM\ Gates$]

$\downarrow$

($RAM\ finds\ 'COMPARE\ Opcode\ 1000'\ at\ 0x1008$)

$Clock\ Cycle\ 3\ (T3): THE\ PAYLOAD\ LATCH$

$[\ Memory\ Data\ Reg\ (MDR)\ ]\ <--\ [DATA\ BUS\ carries: 1000]\ --[\ RAM\ Output\ ]$

$$(CU\ Signal\ 3: MDR_IN_EXT\ latch)\begin{bmatrix} Memory\ Data\ Register_INPUT_External \\ [Here\ External\ means\ External\ Data\ Bus] \\ \begin{bmatrix} Input\ Data\ Signal\ from\ External\ Data\ Bus \\ connected\ with\ RAM \end{bmatrix} \end{bmatrix}$$

$Now\ Holds: 1000\ (The\ COMPARE\ Opcode\ Cargo)$

$Clock\ Cycle\ 4\ (T4): INTERNAL\ ROUTING\ (The\ Decode)$

$[\ Memory\ Data\ Reg\ (MDR)\ ]\ Holds: 1000$

$\downarrow$

$(CU\ Signal\ 4: MDR_OUT)$

$(Flows\ over\ internal\ CPU\ wiring)$

$[\ Instruction\ Reg\ (IR)\ ]\ (CU\ Signal\ 5: IR_IN\ latch)$

$Now\ Holds\ Opcode: 1000\ (COMPARE)$

$*\ This\ complete\ sequence\ is\ exactly\ why\ any\ incoming\ information\ MUST\ go\ through\ the\ MDR!\ *$

==========

$C++\ TO\ HARDWARE\ MAPPING\ (The\ Setup\ Phase)$

==========

$\underline{C++\ Code:}$	$\underline{Hardware\ Execution:}$
$int\ i\ =\ 0;\quad ----->$	$CU\ extracts\ immediate\ '0'\ from\ Instruction$
	$and\ loads\ it\ into\ REG\ 1.$
	$(REG\ 1\ is\ now\ officially\ 'i')$

$n = 1;$ $-----> CU\ extracts\ immediate\ '1'\ from\ Instruction$

$and\ loads\ it\ into\ REG\ 2.$

$(REG\ 2\ is\ now\ officially\ 'n')$

$Visual\ Scenario\ B: Evaluating\ Iteration\ 1\ (i = 0, n = 1)$

The CPU must determine if $i < n$. *It uses subtraction* $(0 - 1 = -1)$.

Because the result is a negative number, the $COMPARE$ *instruction*

does NOT escape the loop.

```
==============================================
 VISUAL B: EVALUATING ITERATION 1 (i = 0, n = 1)
=====================================================

Execution Phase of COMPARE (Opcode 1000 sitting in IR)

 [ REG 1 ('i') ] Holds: 0                    [ REG 2 ('n') ] Holds: 1
    |                                                  |
    \-----(Internal CPU Wires A & B) ------/
                    |
                    v
          [ ALGORITHM AND LOGIC UNIT ]
          (CU Signal 6: "ALU_SUBTRACT")
               Performs: 0 - 1 = -1
                    |
                    | (Result is Negative)
                    v
      [ ZERO FLAG WIRE ] <--- Stays OFF (No electricity flowing)
```

Decision Phase of JUMP_IF_EQUAL (Opcode 1001, Target 0x101C sitting in IR)

[***Instruction Reg (IR)***] ***Holds Opcode: 1001 (JUMP_IF) Target: 0x101C***

↓

[***CONTROL UNIT***]

--- ***7. Checks ALU: Is ZERO FLAG ON? --> (NO! It is OFF.)***

--- ***8. The Condition is FALSE. The CU will NOT jump.***

--- ***9. Let the Program Counter proceed normally.***

[***Program Counter (PC)***] ***Proceeds sequentially: 0x100C + 4 = 0x1010.***

Result: The CPU enters the BODY of the loop.

Again – ITERATION 2

Address	***Raw Hex***	***Raw Binary Blueprint***
0x1008:	**0x8120**	**[1000] [0001] [0010] [0000]**
0x100C:	**0x901C**	**[1001] [0000] [00011100]**

Step 1: The COMPARE Execution (0x1008)

The PC is at 0x1008. The CU fetches the machine code from this address in the TEXT segment.

The Machine Code Representation:

```
Address     Raw Hex     Raw Binary Blueprint
--------    --------    --------------------------------
0x1008:     0x8120      [1000] [0001] [0010] [0000]
                         |      |      |      |
                 Opcode--/      |      |      \-- Padding (Unused)
             (COMPARE)          |      |
                                |      \-- Input B: REG 2 ('n')
                                |
                                \-- Input A: REG 1 ('i')
```

What Happens Physically:

1. *The CU reads* 1000 (*COMPARE*) *and knows it needs the ALU*.
2. *The CU routes REG* 1 (1) *and REG* 2 (1) *directly into the ALU*.
3. *The ALU performs the subtraction*: $1 - 1 = 0$.
4. ***THE CRITICAL EVENT***: *Because the result of the math is exactly zero, the ALU sends high voltage down a specific wire called the* ***Zero Flag***. *The Zero Flag is now* ***ON***.

Step 2: ***The JUMP_IF_EQUAL Decision*** ($0x100C$)

The PC automatically steps forward to $0x100C$. *The CU fetches the next instruction from the TEXT segment. This is the eject button.*

The Machine Code Representation:

```
Address     Raw Hex     Raw Binary Blueprint
--------    --------    --------------------------------
0x100C:     0x901C      [1001] [0000] [00011100]
                         |      |      |
                 Opcode--/      |      \-- Operand: 0x1C (Target Address)
         (JUMP_IF_EQUAL)        |
                                |
                                \-- Register: Unused (Padding)
```

What Happens Physically:

1. *The CU reads* 1001. *The microcode recipe for this Opcode tells the CU*: *"Look at the ALU's Zero Flag wire. If it has voltage (is ON), overwrite the Program Counter with the Operand."*

2. *The CU checks the Zero Flag wire.* ***It is ON***. *(During Iteration* $1, 0 - 1 = -1$, *so the flag was OFF, and the CU ignored this step!).*

3. *Because the flag is ON, the condition is TRUE.*
4. *The CU grabs the Operand from the machine code.* 00011100 *in binary is* 28 *in decimal, which translates to* $\mathbf{0x1C}$ *in hexadecimal.*

5. *The CU takes the Base Address* $(0x1000)$ *and adds the* $0x1C$ *to get the final escape coordinate*: $\mathbf{0x101C}$.

6. *The CU violently shoves* $0x101C$ *directly into the Program Counter (PC).*

==========

THE HARDWARE ESCAPE SEQUENCE

==========

[***ALGORITHM AND LOGIC UNIT***]

Math: $\mathbf{1 - 1 = 0}$

Result: [***ZERO FLAG WIRE***] = ***ON***!

[***CONTROL UNIT***]

Reads Opcode $'\mathbf{1001}'$ (***Jump if Zero Flag is ON***)

Senses the Zero Flag is ON.

Extracts target $'\mathbf{0x101C}'$ ***from the machine code***.

↓ (***CU routes target address to PC***)

[***Program Counter*** (***PC***)] ***OVERWRITTEN***! ***Now holds***: $\mathbf{0x101C}$

$\boldsymbol{Step\ 3: The\ Result\ (0x101C)}$

$The\ current\ Fetch - Execute\ cycle\ is\ complete. The\ CPU\ takes\ a\ breath,$

$ticks\ the\ clock, and\ looks\ at\ the\ Program\ Counter\ to\ find\ out\ where\ to\ go\ next.$

$Because\ the\ PC\ now\ holds\ 0x101C, it\ completely\ skips\ over\ 0x1010\ (the\ PRINT\ statement)$

$and\ 0x1014\ (the\ i++\ statement). The\ CPU\ fetches\ whatever\ machine\ code$

$is\ sitting\ at\ 0x101C\ (ZONE\ 5: OUTSIDE).$

$The\ loop\ has\ been\ successfully, physically\ broken\ by\ the\ hardware!$

$\boldsymbol{Visual\ Scenario\ C: Escaping\ the\ Loop\ (i = 1, n = 1)}$

$The\ loop\ body\ has\ completed.\ i\ was\ incremented, so\ REG\ 1\ now\ holds\ 1.$

$The\ unconditional\ JUMP\ (1111) teleported\ the\ PC\ backward\ to\ 0x1008.$

$The\ condition\ check\ (i < n)\ repeats.$

$1 - 1 = 0. Because\ the\ result\ is\ zero, the\ ALU\ turns\ on\ the\ ZERO\ FLAG\ WIRE.$

$This\ wire\ tells\ the\ Control\ Unit\ (CU)\ that\ it\ is\ time\ to\ escape!$

==========

$\boldsymbol{VISUAL\ C: ESCAPING\ THE\ LOOP\ (i = 1, n = 1)}$

==========

$\boldsymbol{Execution\ Phase\ of\ COMPARE\ (Opcode\ 1000\ in\ IR)}$

$[\ \boldsymbol{REG\ 1\ ('i')}\ ]\ \boldsymbol{Holds: 1}$ $\qquad\qquad$ $[\ \boldsymbol{REG\ 2\ ('n')}\ ]\ \boldsymbol{Holds: 1}$

$|$ $\qquad\qquad\qquad

$Decision\ Phase\ of\ JUMP_IF_EQUAL\ (Opcode\ 1001, Target\ 0x101C\ sitting\ in\ IR)$

$[\ Instruction\ Reg\ (IR)\ ]\ Holds\ Opcode: 1001\ (JUMP_IF)\ Target: 0x101C$

$\downarrow$

$[\ CONTROL\ UNIT\ ]$

$---\ 10. Checks\ ALU: Is\ ZERO\ FLAG\ ON?\ -->\ (YES!\ It\ is\ ON.)$

$---\ 11. The\ Condition\ is\ TRUE. Execute\ the\ escape!$

$---\ 12. CU\ routes\ the\ target\ address\ directly\ from\ the\ IR.$

$\downarrow$

$(Internal\ CPU\ Wire\ carrying\ target: 0x101C)$

$\downarrow$

$[\ Program\ Counter\ (PC)\ ]\ OVERWRITTEN!\ (CU\ Signal\ 13: PC_IN\ Latch)$

$Now\ holds: 0x101C$

$Result: On\ the\ very\ next\ clock\ cycle, the\ CPU\ entirely\ skips\ the\ loop\ body$

$and\ teleports\ to\ 0x101C!$